

JOSÉ ANTONIO FUENTES SANTIAGO

PROGRAMMING GUIDE FOR A ZOMBIE APOCALYPSE



Single Player

Software Development Book

Programming Guide for a Zombie Apocalypse (Demo)

José Antonio Fuentes Santiago

Contents

Block 1. Programming fundamentals	1
Chapter 1. Comments. Writing down what you need	3
Single-line comments	3
Multi-line comments	3
JSDoc comments	4
Disabling code with comments	4
What deserves a comment	4
Survival exercises	5
Chapter 2. Variables and constants. Organizing your backpack	7
Variables	7
Constants	8
Good practices for clean code	9
Avoid magic numbers	10
Survival exercises	11
Chapter 3. Primitive data types. Sorting the supplies	13
Numeric types	13
Strings	15
Booleans	16
When there are no resources: <code>null</code> and <code>undefined</code>	16
Union types	17
The <code>any</code> type	18
The <code>unknown</code> type	19
Other primitive types	20
Survival exercises	20
Chapter 4. Operators. How to read the signs around you	22
Arithmetic operators	22
String operators	23
Comparison operators	24
Logical operators	26
Unary operators	30
Compound assignment	31
Survival exercises	32

To be continued...

36

Block 1. Programming fundamentals

Now...

First, it was an alert on everyone's phones.

Then, the screams in the street.

After that, silence.

After weeks of rumors, it took only a few hours for the world you knew to stop working. The streets are empty, the phones don't ring, and something is moving on the other side of the city... something that doesn't walk like you. Something that doesn't think like you. Something that never stops...

The alert was clear: get out of the most populated areas and find shelter far from the crowds.

You're still at home. You look around and see canned food, your old notes, a flashlight that's half dead, and an old laptop that, by some miracle, still works. It's now or never. You're going to need knowledge, ingenuity, and a well-packed backpack.

You don't know how long you'll be out there. But you know one thing: you'll survive. And to do that, you need to understand how this new world works.

Today you'll learn to go through everything in your backpack. You'll write down what matters, sort the supplies, and organize your resources, and then you'll read the signs around you and make your first hard decisions. There will even be time to reorganize your tools and add the finishing touches before you walk out the door.

In this block...

You'll see how to get your backpack ready to leave the safety of your home... and face

the outside world.

- **Chapter 1. Comments:** Learn to document your code as if it were your survival journal.
- **Chapter 2. Variables and constants:** Find out how to store and organize information in your program.
- **Chapter 3. Primitive data types:** Get to know the basic data types and how to use them.
- **Chapter 4. Operators:** Master the basic operations for working with data and making decisions.
- **Chapter 5. Conditionals:** Build the logic for making decisions in your code.
- **Chapter 6. Loops:** Learn to repeat actions efficiently.
- **Challenge 1: 7-day survival simulator:** Put everything you've learned to the test.
- **Block 1 summary:** Go over the key concepts.

Chapter 1. Comments. Writing down what you need

Before going outside to face the creatures lurking around every corner of the city, any survivor with a bit of sense grabs a notebook and starts writing things down: escape routes, how the tools that might keep you alive work, the zombie behavior patterns you've noticed, and so on. Any one of those notes could determine whether you survive or turn into one of those things.

Comments are short notes that help you keep track of what each part of the code does, like jotting reminders on the cover of your field notebook. The transpiler, interpreter, or compiler (depending on the type of programming language, as we saw earlier) ignores them, so they're only there for you.

Single-line comments

Single-line comments start with `//`, and everything you write from there to the end of the line is part of the comment.

```
// Declare our starting resources.  
let totalRations: number = 24; // Total rations available.  
let rationsPerDay: number = 3; // Daily consumption per person.  
  
// Calculate how many days the provisions will last.  
let daysOfProvisions: number = totalRations / rationsPerDay;
```

Multi-line comments

You can also write comments that span several lines. This kind of comment starts with `/*` and ends with `*/`. Everything between the opening and the closing is treated as part of the comment.

```
/*  
 * Rations needed for the escape:  
 * - You eat 3 rations a day.  
 * - We add an extra day in case something goes wrong.  
 */  
let escapeDays: number = 4;  
let escapeRations: number = (escapeDays + 1) * 3;
```

JSDoc comments

There's a special kind of comment you can use with TypeScript and JavaScript. Most code editors show it as contextual help, and documentation tools can export it to different formats. These comments are called JSDoc, and they follow a set of rules that help you document your code properly.

They start with `/**` and end with `*/`, like any other multi-line comment. In TypeScript you don't need to repeat the type inside the comment, because it's already in the code.

```
/**
 * Checks whether a zone is safe.
 * @param coordinates Coordinates of the zone to check.
 * @returns Whether the zone is safe or not.
 */
function isSafeZone(coordinates: string): boolean {
    // Simulation: zones that start with "A" are safe.
    return coordinates.startsWith("A");
}
```

Disabling code with comments

Comments also let you switch off a line without deleting it, which is known as commenting it out. That comes in handy when you're trying something out and want to be able to go back quickly:

```
let availableRations: number = 12;
// availableRations = availableRations - 3;
console.log(availableRations); // 12: the subtraction is disabled.
```

You'll see this a lot in this book, especially to show code that would cause an error.



Note from the cabin

In most editors you can comment or uncomment the selected lines with `Ctrl + /` (`Cmd + /` on macOS).

What deserves a comment

Compare these two comments:

```
let travelDays: number = 4;  
  
// We add 1 to the days.  
let daysWithMargin: number = travelDays + 1;  
  
// One extra day in case the route is blocked and we have to take a detour.  
let plannedDays: number = travelDays + 1;
```

The first one repeats what the code already says: anyone can see it adds 1. The second one explains **why** it adds it, which is the one thing the code can't tell you. When you're not sure whether to write a comment, ask yourself whether it answers a "why." If it only describes the "what," you can probably do without it.



Survivor's tip

If everything is well organized and labeled, you don't need to look inside every bag to know what's in it. Programming works the same way: if you pick good names, structure things well, and keep the code clear enough, you'll hardly need comments. Even so, there are times when comments help, like when you need to explain a complex decision or warn about a hidden trap in the code.

Survival exercises

Exercise 1.1: Documenting your gear

You've been at home for hours, with the blinds down and the silence broken only by distant moans and shuffling footsteps that are slowly getting closer. Outside, the city is no longer safe. The creatures roam the streets, and soon you'll have to leave before they find out where you are. Before you take that step, you open your field notebook, because it's time to record what you're carrying in your backpack. Packing isn't enough. You need to know exactly what you have, and every item counts.

Your mission: create a TypeScript file (remember, with the `.ts` extension) that documents your survival gear:

- Copy the four variables below and, above each one, write a single-line comment explaining what that item is for.
- At the end of the file, write a multi-line comment with your strategy for the next few days.

- The flashlight has run out of batteries: disable its line by commenting it out and check that the file still runs.

```
let backpack = "40L tactical backpack";  
let waterLiters = 2.5;  
let foodRations = 8;  
let flashlight = true;
```

Chapter 2. Variables and constants. Organizing your backpack

The silence of the house is broken by a distant moan drifting through the night air. There's no time to lose. Your survival backpack has to be in order before those sounds get any closer, and every resource you pack needs a clear label with its name: "water," "food," "map," "ammo." That way, when you need something in a hurry while you're on the run, you'll know exactly where to find it without making any unnecessary noise.

In programming, those labels are the names of the **variables** and **constants** that let you organize your resources and find them quickly.

Variables

To start getting organized, you grab your sticky notes and write the name of a supply from your backpack on each one. A variable is like one of those sticky notes: you can peel it off and stick it on something else when you need to. Say you have a bottle with the "water" note on it. If the bottle runs out, you can move the label to another bottle of water. The contents have changed, but the label is still useful, as long as it holds the same type of data.

This is the format for declaring a variable:

```
// The "=" symbol assigns a value to the variable.  
let variableName: type = value;  
  
// You can declare it without an initial value, but TypeScript won't let  
// you use it until you assign one.
```

The `type` says what kind of data the variable will hold: a number, a piece of text... You'll look at types properly in chapter 3. For now, all you need to know is that `number` means a number.

By convention, variable names are written in `lowerCamelCase`: the name starts with a lowercase letter, and if it has several words, each word after the first starts with a capital letter.

```
let remainingWaterLiters: number = 5;  
  
// This variable has no initial value.
```

```
let daysOfFood: number;  
  
// You can change the value of any variable once it's declared.  
  
remainingWaterLiters = 4;  
daysOfFood = 2;
```

Besides the convention, there are a few rules that aren't optional. If you break them, TypeScript won't let you go on:

- A name can't start with a number: `2liters` doesn't work, but `liters2` does.
- It can't contain spaces or hyphens: `liters water` and `liters-water` don't work.
- You can't use the language's reserved words, like `let`, `const`, `if`, or `class`.
- Names are case-sensitive: `water` and `Water` are two different variables. It's better not to have both.

Constants

Imagine you write an object's name directly on it with a permanent marker. Once you've named it, that name and those contents won't change for the whole expedition. That way you avoid mix-ups and make sure the important things are always clearly identified. Constants work in a similar way: once they're initialized, you can't change their value for as long as the application is running.

The format for constants is very similar to the one for variables, except that here you have to assign an initial value:

```
const CONSTANT_NAME: type = value;
```

Constant names are written in uppercase, with `_` to separate the words. That way, when you read the code, you can tell at a glance the fixed values that set the rules of your program (the daily water consumption, the capacity of the backpack) from the ones that keep changing:

```
const LITERS_PER_DAY = 2;  
const HELP_MESSAGE = "SOS";  
  
// If we try to change the value of a constant, we get an error.  
// LITERS_PER_DAY = 3;  
// Type error: Cannot assign to 'LITERS_PER_DAY' because it is a
```

```
// constant.
```



Note from the cabin

`const` is actually used in two ways. The first is the one you've just seen: global configuration values, which are set at the start of the program and affect all of it. They're written in UPPERCASE. The second is for values that are calculated at a given moment and don't change afterward, like the number of days your water will last. Many teams declare those with `const` too, but in lowerCamelCase: `const daysOfWater = waterLiters / LITERS_PER_DAY; .`

To keep things simple at the start, in this block we'll use `let` for everything that isn't a global configuration value, and `const` in UPPERCASE only for those values. When you start writing functions, you'll see the second one.



Survivor's tip

Use only letters of the English alphabet, digits, and underscores for the names of everything you define in your program: variables, constants, functions, enums, classes, and so on. Some languages accept other characters (ñ, á, ü, and so on), but they can cause compatibility problems when you share your code with other survivors or use tools from different systems. For the same reason, don't mix two languages in your names. It's like using universal signals instead of local dialects, so everyone can understand you wherever they come from.



Note from the cabin

JavaScript also has the `var` keyword for declaring variables. It isn't recommended, though, because it can cause unexpected behavior related to scope and to redeclaring variables. Whenever you can, use `let` or `const` to keep your code safer and clearer.

Good practices for clean code

On an expedition, being organized and clear can determine whether you make it back or get lost. In programming, following good practices helps you keep your code readable and working:

- Name your variables and constants clearly. Use descriptive names and avoid confusing abbreviations.
- Keep your naming style consistent: lowerCamelCase for variables and UPPER-CASE_WITH_UNDERSCORES for constants.
- Don't reuse variables for different purposes. Each label should have a single use, so `daysOfWater` shouldn't end up storing the number of batteries.

Here's an example of names that tell you very little:

```
let l = 2;  
let r = 4;  
let c = r / l;
```

That could mean anything, and you'll waste valuable time figuring it out. The following code, on the other hand, is easy to follow and tells you right away what each variable represents:

```
// LITERS_PER_DAY and remainingWaterLiters come from the earlier examples.  
let daysOfWater: number = remainingWaterLiters / LITERS_PER_DAY;
```

Avoid magic numbers

When you write code, you sometimes end up with numbers that work fine but whose meaning isn't obvious. For example, imagine you calculate the weight of your backpack like this: `let totalWeight = 8.2 + 3.5 + 1.8;`. It works, but if you come back to that code a few days later, will you remember what each number stands for? Are they kilograms of water, food, ammo? Why those specific values?

These literal values with no context are known as **magic numbers**: numbers that appear directly in the code with no explanation of what they mean. They make the code harder to read and maintain, because whoever reads it (including your future self) has to guess what they represent.

Look at these two versions of the same calculation:

Code with magic numbers:

```
// In a real program, backpackWeight would come from other calculations.  
let backpackWeight: number = 11.5;  
let remainingCapacity = 15 - backpackWeight;
```

```
// Backticks let you put values inside the text (chapter 3).  
console.log(`You can carry ${remainingCapacity} kg more`);
```

Code without magic numbers:

```
const MAX_BACKPACK_CAPACITY_KG = 15;  
  
let remainingCapacityKg = MAX_BACKPACK_CAPACITY_KG - backpackWeight;  
  
console.log(`You can carry ${remainingCapacityKg} kg more`);
```

The difference is clear. In the second case, when you read your code again you'll understand right away what the value that used to be 15 stands for. And if you need to change the capacity of the backpack, you only change the value of the constant in one place, and every calculation that uses it will use the new value.



Survivor's tip

Not every number is a magic number. Values that are obvious, like 0, 1, 2 (to double something), 100 (for percentages), or -1 (to mean “not found”), are usually clear on their own. Use your judgment: if a number needs context to be understood, turn it into a constant with a descriptive name.

Survival exercises

Exercise 2.1: Organizing your inventory

The growls sound closer now, and you know you can't afford a single mistake. You open the backpack on the table with shaking hands and see that it has a fixed amount of space and a maximum weight you can't go over without breaking it. Within those limits, though, you know your inventory will keep changing: one day you'll carry more water, another day less food, and the weight will vary with every item you add or use up while you run from the zombie hordes.

In your field notebook, with the flashlight lighting the pages, you decide to record which things never change and which ones change over time. That way you'll always know how to organize your resources, so you aren't left defenseless in the middle of your escape when the horde shows up.

Your mission: declare variables and constants to organize your survival inventory.

- Define constants for:
 - The capacity of the backpack, which is 40 liters
 - The maximum weight of the backpack, which is 15 kilograms
- Declare variables for the resources that can change:
 - The number of liters of water, with an initial value of 3.5 liters.
 - The current number of food rations, and then assign it a value of 12 rations.
 - The current weight, with an initial value of 8.2 kilograms.

Chapter 3. Primitive data types. Sorting the supplies

Not all supplies are the same. Some are liquid and others solid; some feed you and others help you find your way. It's the same in programming: every piece of data has a **type**, and as a survivor you need to know them as well as you know your compass or your knife.

To say what type a piece of data has, you write `: type` right after the name. For example:

```
let zombies: number = 14;
let survivorName: string = "Marcos";
```

Once a variable has a type, TypeScript makes sure you don't put anything else in it. If you try, the metal detector beeps:

```
let zombiesDetected: number = 14;
// zombiesDetected = "fourteen";
// Type error: Type 'string' is not assignable to type 'number'.
```

TypeScript's error messages can be hard to read at first. Appendix A covers the ones you'll see most often, explained with examples.

You don't always need to write the type. If you give the variable an initial value, TypeScript figures it out on its own. This is called **type inference**:

```
let sheltersVisited = 3; // TypeScript infers that it's a number.
// sheltersVisited = "three";
// Type error: Type 'string' is not assignable to type 'number'.
```

In this book we'll write the type almost every time, so you have it in front of you while you learn. In your own code you can leave it out when the initial value already makes it clear.

Numeric types

Numeric data types represent the resources you can count or measure. They tell you how much you have, how much you need, and how much you can risk.

Integers are the right fit for counting objects, steps, or zombies.

```
// An exact number. You don't keep half batteries.
let batteries: number = 24;
```

Decimal numbers are for more precise measurements: distance, time, temperature, weight.

```
// Precision matters here.  
let temperature: number = -15.7;
```



Survivor's tip

In code, decimals are written with a period (.), as in 2.5 , and large numbers are written without commas: 1500 , not 1,500 . Most programming languages (TypeScript included) use the period as the decimal separator.

Although we've explained these two kinds of numbers separately, TypeScript makes no distinction between integers and decimals. That's why there's only one type, `number` , for both.

Since `number` covers both, TypeScript won't stop you from storing 2.5 batteries. Deciding whether a value can have decimals is up to you.

Decimals have another surprise in store. Add two amounts of water:

```
let bottleWater: number = 0.1;  
let canteenWater: number = 0.2;  
console.log(bottleWater + canteenWater); // 0.30000000000000004
```

That isn't a bug in TypeScript. Almost every language stores decimals in binary, and some of them, like 0.1 , can't be represented exactly. To display the result, round it with `toFixed()` and say how many decimal places you want (you'll look at it in depth in chapter 8):

```
console.log((bottleWater + canteenWater).toFixed(2)); // "0.30"
```



Note from the cabin

When you work with numbers, a mysterious value can turn up: `NaN` (Not a Number). It's like finding a can in your backpack that looks like food, but when you open it, it's empty or full of something you can't eat. `NaN` shows up when you try an impossible math operation, like dividing 0 by 0 or turning a word into a number. If you see `NaN` , don't trust that resource. It means something went

wrong in your calculations, and you could run out of provisions when you need them most. Always check your math and make sure there are no empty cans in your inventory.

Strings

Strings are like the messages you leave written on walls or the notes you jot down in your survival notebook. A string (`string`) is a sequence of characters, anything from a single word to whole paragraphs of information.

In the post-apocalyptic world, these strings can hold:

- **Place names:** “North Shelter”, “Abandoned Hospital”
- **Emergency messages:** “SOS - We need medical supplies”
- **Coordinates:** “Grid 47-B, Industrial Sector”
- **Inventories:** “3x Cans, 2x Medicine, 1x Radio”

```
let safeDestinationName: string = "Tablillas Dam";  
let message: string = "Hostile group spotted to the south";  
let coordinates: string = "Lat: 40.7128, Lon: -74.0060";
```

You need strings to communicate with other survivors and to keep a record of what you find. Without them, you couldn't document your discoveries or give an exact location over the radio.

In TypeScript, there are three ways to write a string:

```
// With single quotes.  
let warning: string = 'Watch out! Zombies nearby.';  
  
// With double quotes.  
let notice: string = "Safe shelter found";  
  
// With template literals, which use the ` symbol.  
// They let you embed variables, constants, and other code with the  
// following format: `${codeToInclude}`  
let zone: string = "Shopping Mall";  
let sector: number = 7;  
let report: string = `Exploring ${zone}, sector ${sector}`;
```

Template literals can also span several lines. With regular quotes, you have to write

`\n` to start a new line:

```
let patrolReport: string = `Zone: north
Zombies: 3
Status: under control`;

let radioReport: string = "Zone: north\nZombies: 3\nStatus: under control";
// Both variables contain exactly the same text.
```



Survivor's tip

In most languages, strings go between quotes. Make sure you close those quotes properly, because if you forget one, your message can get lost in the jungle of code.



Note from the cabin

Throughout the guide, for compatibility with other languages, we use double quotes `"` whenever we aren't going to include the value of a variable or constant.

Booleans

The `boolean` type has only two values: **true** (`true`) and **false** (`false`). That's what lets you make decisions. March on or stay? Light the fire or wait?

```
let isSafeZone: boolean = true;
let stormApproaching: boolean = false;
```



Survivor's tip

Never assume something is true by default. Check your conditions, or you could take a zone full of traps for a safe one.

When there are no resources: `null` and `undefined`

`null`

When you're surviving, there are times when you reach into your backpack and there's nothing left. No water, no food, no ammo. In programming, those moments are represented with `null`.

Having a `null` value is like looking inside your backpack and seeing that a compartment is empty, although you know the compartment is there. You kept something in it before, but now there's nothing. It's an absence on purpose, because you're the one who decided to leave it empty.

```
// You've decided to leave the battery pocket empty.
```

```
let batteryPocket = null;
```

```
console.log(batteryPocket); // null
```

undefined

`undefined` represents the absence of a value because the variable hasn't been initialized. The JavaScript engine assigns it when you declare a variable without giving it a value, or when you access a property that doesn't exist. Other languages have similar concepts, but the way `undefined` behaves is specific to JavaScript, so it's worth knowing its quirks.

```
// You declare the variable, but you haven't given it a value yet.
```

```
let nextDestination;
```

```
console.log(nextDestination); // undefined
```



Survivor's tip

Before you go out exploring, check your backpack carefully and make sure you have no empty compartments (`null`) or forgotten ones (`undefined`). If you try to use something that isn't there, you could be in serious trouble when you need it most.



Note from the cabin

Remember that `null` and `undefined` are different values. Use `null` when you want to say that something is empty on purpose, and avoid using `undefined` yourself, since the language already assigns it when a variable hasn't even been initialized. Running out of resources isn't the same as forgetting to pack them.

Union types

You turn on the radio and all you hear is static. You want to store the last message you received, but none has come in yet. A variable of type `string` doesn't accept `null`, so

you have to tell TypeScript that it may be empty. You do that with a vertical bar (`|`), which reads as “or”:

```
// You haven't received any radio messages yet.  
let lastRadioMessage: string | null = null;  
  
// Hours later, someone answers.  
lastRadioMessage = "This is north shelter. Does anyone copy?";
```

This is called a **union type**: the variable accepts any of the types you separate with `|`. You'll run into unions again later on.

The `any` type

Imagine you find a closed box while you're out exploring. You don't know what's inside. It could be food, medicine, ammo, or even something dangerous. TypeScript has a special type called `any` that works like that mystery box: it can hold any type of data.

```
let foundBox: any = "Emergency radio";  
console.log(`Initial contents: ${foundBox}`);  
  
// Later you find out it can also be a number.  
foundBox = 42;  
console.log(`Now it contains: ${foundBox} batteries`);  
  
// Or even a boolean.  
foundBox = true;  
console.log(`Does it work? ${foundBox}`);
```

The `any` type is useful when:

- You're working with old code that has no types defined.
- You receive data from external sources (like radio messages) and don't know what to expect.
- You need a temporary solution while you figure out the exact type.

Even though this type exists, you should use it with great care. It's like carrying an unlabeled box in your backpack: it works, but you lose the advantage of knowing what you have. TypeScript can't help you catch errors if you overuse `any`. Whenever you can, specify the type of your data. Your future self will thank you.

**Note from the cabin**

When you come across mystery data in your code, you can use the `typeof` operator like a flashlight to inspect what kind of supply you really have. It's especially useful when you work with variables of type `any` or receive data from external sources.

```
let unknownBox: any = "Medicine";
console.log(typeof unknownBox); // "string"

unknownBox = 25;
console.log(typeof unknownBox); // "number"

unknownBox = true;
console.log(typeof unknownBox); // "boolean"

unknownBox = null;
// Watch out for this trap!
console.log(typeof unknownBox); // "object"
```

The `typeof` operator returns a string that identifies the type: `"string"`, `"number"`, `"boolean"`, `"undefined"`, `"object"`, `"function"`, `"symbol"`, or `"bigint"`. Keep in mind that `typeof null` returns `"object"` for historical reasons in JavaScript, so don't rely on this check alone when you're looking for null values.

The unknown type

If you really don't know what type a piece of data will have, there's a safer alternative to `any`: `unknown`. It accepts any value, just like `any`, but it won't let you use that value until you check what it is:

```
let sealedBox: unknown = "Emergency radio";

// let label: string = sealedBox;
// Type error: Type 'unknown' is not assignable to type 'string'.

if (typeof sealedBox === "string") {
    // Here TypeScript already knows it's a string.
    console.log(`The box contains: ${sealedBox}`);
}
```

With `any`, you open the box and stick your hand in without looking. With `unknown`, you have to look inside before you touch anything.



Note from the cabin

Don't expect to find these types in every language. `undefined` is specific to JavaScript (and so to TypeScript), and `unknown` only exists in TypeScript. Most languages get by with a single value for saying "there's nothing here," like `null` in Java, `None` in Python, or `nil` in Go, and they don't distinguish between something left empty on purpose and something nobody ever got around to filling.

Other primitive types

There are two other primitive types that you won't use in this book: `bigint`, for huge integers, and `symbol`, for creating unique identifiers. If you ever need them, at least you'll know they exist.

Survival exercises

Exercise 3.1: Classifying supplies

The first shelter you've found is little more than a maintenance shed forgotten on the outskirts. There isn't much room, but it's solid enough to spend the night in and stay clear of the groups of zombies that never stop walking.

With your flashlight you inspect every corner: a little canned food, an old radio that might still work, and fresh footprints that may or may not be human.

In your field notebook you decide to classify the most important data: the name of the shelter, the temperature, whether it's safe, and the last radio message you received. If you don't record it clearly, you could forget details you'll need when the time comes to decide.

Your mission: create variables with different data types for your inventory:

- Name of the shelter: a string.
- Current temperature: a decimal number.
- Is it a safe zone? A boolean.

- Last radio message: a string, or `null` if there's no contact. Declare it as `string | null` and start with no contact.
- Print the status of the shelter to the console with all four values.
- Also print, with `typeof`, the type of the temperature.
- Finally, simulate a message coming in: store some text in the radio variable and print it again.

```
// An example of how to start:
```

```
let shelterName: string = "Alpha Shelter";
```

```
console.log("=== SHELTER STATUS ===");
```

Chapter 4. Operators. How to read the signs around you

In an extreme situation, your brain never stops analyzing your surroundings: it calculates how much you have left, compares risks, and combines signals to make decisions. In programming, you do that with **operators**.

Operators are symbols that let you **transform and evaluate data**: add, subtract, compare, check conditions... In other words, they're like your senses, your knowledge, and your judgment working together to keep you alive.

Arithmetic operators

When you need to know how many days you can hold out, how far you still have to go, or whether the supplies will last the whole week, you're doing **arithmetic**.

```
let totalFood: number = 50; // Rations available.
let dailyConsumption: number = 3; // Rations per day.
let totalMetersToDestination: number = 15000; // Meters to the shelter.
let metersTraveled: number = 8500; // Meters already walked.

let survivalDays: number = totalFood / dailyConsumption;
let remainingDistance: number = totalMetersToDestination - metersTraveled;

console.log(`You can survive ${survivalDays} days`);
console.log(`You have ${remainingDistance} meters left to go`);
```

If you run this code, you'll see that you can survive `16.666666666666668` days. That's correct, but nobody counts days like that. As you saw in chapter 3 with the water, `toFixed()` lets you keep only the decimals you want: `survivalDays.toFixed(1)` gives `"16.7"`.

These are the basic arithmetic operators in most programming languages:

Oper.	Meaning	Precedence	Example
**	Exponentiation	Highest	<code>detection = zombies ** noiseLevel</code>
*	Multiplication	High	<code>energy = steps * caloriesPerStep</code>
/	Division	High	<code>days = rations / dailyConsumption</code>
%	Modulo (remainder)	High	<code>leftover = cans % cansPerBox</code>
+	Addition	Medium	<code>total = waterLiters + teaLiters</code>
-	Subtraction	Medium	<code>missing = goal - progress</code>

Oper.	Meaning	Precedence	Example
-------	---------	------------	---------

Operators with the same precedence are evaluated from left to right. The exception is `**`, which is evaluated from right to left: `2 ** 3 ** 2` is the same as `2 ** 9`.

Modulo (`%`) returns the remainder of a division: `10 % 3` gives `1`, because 10 divided by 3 is 3 with 1 left over. It comes in handy for finding out, say, how many loose cans are left after you fill the boxes. Be careful with negatives: the result takes the sign of the first number, so `-7 % 3` gives `-1`, not `2`.

When you calculate your resources, remember that operator precedence can change the result, so use parentheses to say which operations go first. Otherwise you might think you have more food or water than you really do, and a miscalculation like that can leave you without provisions.

For example:

- The operation `3 + 4 * 12` gives `51`:
 - Because of precedence, the multiplication goes first: `4 * 12 = 48`.
 - Then comes the addition: `48 + 3 = 51`.
- With parentheses, `(3 + 4) * 12` gives `84`:
 - The parentheses give priority to the addition: `3 + 4 = 7`.
 - Then comes the multiplication: `7 * 12 = 84`.

String operators

You can use the `+` operator with strings. In that case it doesn't add anything up. It joins, or **concatenates**, the values:

```

let survivorName: string = "Alex";
let sosMessage: string = "SOS sent by " + survivorName;
// Result: "SOS sent by Alex"

let distance: number = 500;
let report: string = "Shelter is " + distance + " meters away";
// Result: "Shelter is 500 meters away"

```

If you mix numbers and strings with `+`, the number is converted to text:

```
let supplies: number = 15;
let statusMessage: string = "I have " + supplies + " rations";
// Result: "I have 15 rations"

// Watch out for this common mistake!
let result = "5" + 3; // "53" (a string), not 8 (a number).
```



Note from the cabin

JavaScript also lets you use `*`, `/`, or `-` with strings: it tries to convert the string to a number and then does the math. `"10" * 2` gives `20`, but `"ten" * 2` gives the dreaded `NaN`. TypeScript doesn't allow these operations and flags them with a type error (*The left-hand side of an arithmetic operation must be of type 'any', 'number', 'bigint' or an enum type*). If we use TypeScript for anything, it's to make sure you use the right types.



Note from the cabin

To avoid confusion, use **template literals** when you combine text and variables: `'I have ${supplies} rations'`. They're clearer and less error-prone than concatenation.

Even so, you'll see throughout the guide that when the text is very long, we use concatenation so it fits on the page.

Comparison operators

Knowing whether you're better or worse off than yesterday, whether you have the bare minimum, or whether you've reached the limit is part of everyday survival. Your mind is always comparing distances, resources, and threat levels. Are there more zombies than bullets? Has the temperature dropped since dawn? Is the shelter still safe, or has it been compromised?

```
let afternoonTemperature: number = 8;
let arrivalHour: number = 19; // 19:00 (7 p.m.).
let sunsetHour: number = 18; // 18:00 (6 p.m.).
let remainingRations: number = 15;
let criticalLevel: number = 20;
```

```

// Is it safe to go on?
let isCold: boolean = afternoonTemperature < 12;
let needShelter: boolean = arrivalHour >= sunsetHour;
let mustCallForHelp: boolean = remainingRations < criticalLevel;

console.log(`It's cold: ${isCold}`);
console.log(`I need shelter: ${needShelter}`);
console.log(`You must call for help: ${mustCallForHelp}`);

```

These operators return **boolean values**: `true` or `false`.

Oper.	Meaning	Example
<code>==</code>	Equal to (with conversion)	<code>weather == "clear"</code>
<code>===</code>	Equal to (without conversion)	<code>water === 5</code>
<code>!=</code>	Not equal to (with conversion)	<code>zone != "safe"</code>
<code>!==</code>	Not equal to (without conversion)	<code>state !== "danger"</code>
<code>></code>	Greater than	<code>waterLiters > 2</code>
<code><</code>	Less than	<code>zombies < ammo</code>
<code>>=</code>	Greater than or equal to	<code>currentHour >= sunsetHour</code>
<code><=</code>	Less than or equal to	<code>temperature <= 0</code>



Survivor's tip

Don't confuse `=` with `===`. The first one stores a value and the second one compares. If you write `zombies = 0` when you meant to check `zombies === 0`, you aren't asking anything. You've just set the counter to zero.

JavaScript has two kinds of equality and inequality operators, and they work differently. As a survivor, you need to know the difference to avoid mistakes.

Operators with automatic conversion (`==` and `!=`)

```

// This is how JavaScript behaves:
// "1" == 1    gives true: the string "1" is converted to the number 1.
// "3" != 3    gives false: it's converted and 3 != 3 is compared.
// 0 == false  gives true: the value false is converted to 0.

```

Strict operators (`===` and `!==`)

```
// This is how JavaScript behaves:  
// "1" === 1    gives false: different types (string and number).  
// 0 === false  gives false: different types (number and boolean).
```

Strict operators compare both the **value** and the **type** of the data, with no automatic conversions. That makes them more predictable and safer.

If you're wondering why the examples are commented out, there's a good reason: TypeScript won't even let them through. If you compare a string with a number, it tells you the comparison makes no sense (*This comparison appears to be unintentional because the types 'string' and 'number' have no overlap*). Even so, it's worth knowing the difference, because you'll see it in JavaScript code.



Note from the cabin

As a rule, always use `===` and `!==` to avoid unexpected comparisons.

Logical operators

Once you know how to compare, the natural next step is to combine those comparisons to make more complex decisions. A single signal is rarely enough. You weigh several factors, like the weather, your energy, and the terrain, and you combine them in your head. That's what **logical operators** do.

- `!` (not): inverts the boolean value. It has the highest precedence. Example: `!dangerousZone`.
- `&&` (and): it's true when both conditions are true. Medium precedence. Example: `availableWater && safeTerrain`.
- `||` (or): it's enough for one of the two to be true. It has the lowest precedence. Example: `goodWeather || shelterNearby`.

```
// Initial state of the camp.  
let lightSwitch: boolean = true;  
let goodWeather: boolean = false;  
let shelterNearby: boolean = true;  
  
// Flipping the switch changes its state. We can do that with !  
let lightOn: boolean = !lightSwitch;  
// We've turned off the light: now it's false.
```

```

// We can keep marching if the weather is good or there's a shelter
// nearby. Either one of the two is enough to keep walking.
// The following gives true, because there's a shelter nearby.
let keepMarching: boolean = goodWeather || shelterNearby;
// Keep marching (OR): true
console.log(`Keep marching (OR): ${keepMarching}`);

// We can keep marching when the weather is good and there's a shelter
// nearby. We can only go on when both conditions are true.
// The following gives false, because the weather isn't good.
keepMarching = goodWeather && shelterNearby;
// Keep marching (AND): false
console.log(`Keep marching (AND): ${keepMarching}`);

```

To understand how each boolean operation behaves, we use a truth table. It shows every possible combination of inputs and the result you get in each case.

Truth table for the AND operation

(it only returns true when both conditions are true)

Input 1	Input 2	Result
false	false	false
false	true	false
true	false	false
true	true	true

Truth table for the OR operation

(it returns true if at least one condition is true)

Input 1	Input 2	Result
false	false	false
false	true	true
true	false	true
true	true	true

Truth table for the NOT operation

(it inverts the value: what was true becomes false, and vice versa)

Input	Result
false	true
true	false

When you work with logical operators, you won't always use boolean values. You can use any other type of data, which will end up being treated either as `false` (we call these *falsy* values) or as `true` (we call these *truthy* values).

- Falsy values: `null`, `undefined`, `false`, `NaN`, `0`, `-0`, `""` (empty string).
- Truthy values: `true`, any object, any array (even an empty one), any number other than 0, any non-empty string, and the values `Infinity` and `-Infinity`.

When you use values that aren't strictly boolean with the AND and OR operators, the operation returns one of those values:

AND

- If both values are truthy, it returns the second one.
- If not, it returns the first falsy value.

```
// 1 && "Zombie" gives "Zombie": both are truthy, so you get the second.
// 0 && "Zombie" gives 0: it's the first falsy value.
// "Zombie" && 0 gives 0: it's the first falsy value.
// null && 0 gives null: it's the first falsy value.
```

OR

- It returns the first truthy value it finds.
- If both values are falsy, it returns the second one.

```
// 1 || "Zombie" gives 1: it's the first truthy value.
// 0 || "Zombie" gives "Zombie": it's the first truthy value.
// "Zombie" || 0 gives "Zombie": it's the first truthy value.
// null || 0 gives 0: both are falsy, so you get the last one.
```

As before, these lines are commented out because TypeScript warns you that operating on fixed values like `1` or `"Zombie"` makes no sense: the result is known in advance. In a real program you'll use variables.



Note from the cabin

The `!` operator usually inverts a boolean value, but it can work on other kinds of values too:

```
// !""      gives true: a falsy value, negated.  
// !"SOS"   gives false: a truthy value, negated.  
// !0       gives true: a falsy value, negated.  
// !12      gives false: a truthy value, negated.  
// !null    gives true: a falsy value, negated.
```

As with arithmetic operators, you have to keep the precedence of logical operators in mind. Remember that `&&` is evaluated before `||`. If you want one operation to go before another, use parentheses to make the order clear and avoid surprises in your decisions.

This isn't the same:

```
let enoughEnergy: boolean = false;  
keepMarching = (goodWeather || shelterNearby) && enoughEnergy;
```

which lets you keep marching only when you have enough energy and at least one of the other two conditions is true, as this:

```
keepMarching = goodWeather || shelterNearby && enoughEnergy;
```

which lets you keep marching when the weather is good, or when there's a shelter nearby and you also have enough energy, because the `&&` operator takes precedence over `||`.

With the first option, you always need enough energy. With the second one, good weather alone would be enough to keep you marching even if you're exhausted.

**Note from the cabin**

ES2020 introduced a logical operator with a slightly odd name: the **nullish coalescing operator**, written `??`. It works like an OR, but it ignores most `falsy` values and only takes `null` and `undefined` into account. If the first value is one of those two, you get the value that comes after `??`:

```
let availableFuel: number | null = 0;
console.log(availableFuel ?? 50); // 0: it isn't null or undefined.
console.log(availableFuel || 50); // 50: 0 is falsy.
```

This operator makes sure the second value is used only when the first one is undefined or null. It's especially useful with numeric values.

Unary operators

In a tight spot, you sometimes need to make a quick decision about a single resource: add one unit of energy, invert a signal, or check whether you have something in your backpack. **Unary operators** let you change or check the value of a variable with a single operand, like a survival reflex.

The **increment** (`++`) and **decrement** (`--`) operators add or subtract one from a value. In other words, they move the counter one step forward or back.

```
let sodaCans = 5;
sodaCans++; // You've found one more can: now you have 6.
sodaCans--; // You've drunk a can: now you have 5.
```

The **logical NOT** operator (`!`), which you already know from the logical operators, also counts as unary.

Finally, the **unary negation** operator (`-`) changes the sign of a value: a positive one becomes negative and vice versa.

```
let outsideTemperature = 10;
// We change the sign: now it's -10. Watch out for hypothermia!
outsideTemperature = -outsideTemperature;
// Changing the sign of -10 brings the value back to 10.
outsideTemperature = -outsideTemperature;
```

Operator	Meaning	Example
<code>++</code>	Adds one to the value. Increment.	<code>sodaCans++</code>
<code>--</code>	Subtracts one from the value. Decrement.	<code>sodaCans--</code>
<code>!</code>	Inverts the boolean value	<code>!flashlightOn</code>
<code>-</code>	Changes the sign of a number	<code>-temperature</code>

Unary operators let you react fast, but be careful when you use `++` and `--` as a prefix or a suffix while assigning a value. If the operator comes first, the value is incremented or decremented first and assigned afterward:

```
let cans: number = 6;
let remainingCans: number = --cans;
// cans and remainingCans both have the value 5.
```

If the operator comes after the variable, though, the assignment happens first and the value is incremented or decremented afterward:

```
let bullets: number = 6;
let remainingBullets: number = bullets--;
// remainingBullets has the value 6 and bullets has the value 5.
```

Compound assignment

You'll often update a variable based on its own value: you find 3 more rations, or you use up half of your flares. Writing `rations = rations + 3` works, but it repeats the name of the variable. **Compound assignment** operators shorten it: you put the operator right before the equals sign (`=`).

```
let rations: number = 10;
rations = rations + 3; // It can be shortened to:
rations += 3;

let flares: number = 8;
flares = flares / 2; // It can be shortened to:
flares /= 2;

let isSafe: boolean = true;
let hasShelter: boolean = false;
isSafe = isSafe && hasShelter; // It can be shortened to:
isSafe &&= hasShelter;
```

Operator	Same as	Example
<code>+=</code>	<code>x = x + value</code>	<code>rations += 3</code>
<code>-=</code>	<code>x = x - value</code>	<code>water -= 2</code>
<code>*=</code>	<code>x = x * value</code>	<code>flares *= 2</code>
<code>/=</code>	<code>x = x / value</code>	<code>flares /= 2</code>
<code>%=</code>	<code>x = x % value</code>	<code>looseCans %= 6</code>

You'll see them all the time in this book, especially inside the loops in chapter 6.



Note from the cabin

Since ES2021 there are also logical compound operators, `&&=`, `||=`, and `??=`, like the `&&=` in the example.

Survival exercises

Exercise 4.1: Resource calculator

It's been three days since you found an abandoned shelter. Your resources are running low, and you need to figure out whether what you have will be enough to complete the mission.

While you're sorting the supplies in the basement of the shelter, you find marks on the floor: there's something buried nearby that could change your luck. But before you grab the shovel, you need to know exactly how long you can hold out with what you have. The whole dig will take you eight days, and there's no room for miscalculation.

Your mission: do the following calculations in your old field notebook:

- Calculate how many days your water will last and how many days your food will last, each resource separately.
- Calculate how much water and how much food you need in total for the eight days of digging.
- Calculate the difference between what you have and what you need of each resource. If it's negative, you're short.
- Calculate what percentage of your water and food you use each day, to two decimal places.

- Print all the results to the console.

```
// Initial state of the camp after three days of getting organized.  
let totalWaterLiters: number = 45;  
let totalFoodRations: number = 30;  
let missionDays: number = 8;  
let dailyWaterConsumption: number = 2; // Liters per day.  
let dailyFoodConsumption: number = 1.5; // Rations per day.  
  
console.log("=== SHELTER RESOURCE ANALYSIS ===");
```

Exercise 4.2: Early warning system

The storm came without warning. While you were asleep in the shelter, the wind started howling through the cracks in the walls and the temperature plunged. A loud bang wakes you up, and you have to find out what caused it and assess the risks right away.

Over the last few days you've installed several sensors around the shelter, and now your survival depends on reading those signals correctly. The generator has started to cut out, fuel is running low, and visibility has dropped considerably. What you decide in the next few hours could determine whether you make it through the night.

Your mission: define an alert system that checks whether the readings go past the critical thresholds:

- Create a boolean variable for each alert:
 - Dangerous temperature: below -10 °C.
 - Extreme wind: more than 40 km/h.
 - Critical visibility: less than 100 meters.
 - Fuel in reserve: less than 20%.
 - High-risk hours: from 21:00 (9 p.m.) on.
- Figure out whether at least one alert is active.
- Figure out whether you have to evacuate immediately: when the temperature is dangerous and the wind is extreme at the same time, or when the fuel is in reserve during high-risk hours.
- Print the value of each alert and the two final results to the console.

```
// Current sensor readings during the storm.  
let temperature: number = -12;  
let windKmh: number = 35;
```

```
let visibilityMeters: number = 150;
let fuelLevel: number = 18; // Percentage.
let currentHour: number = 22;

console.log("=== EMERGENCY ALERT SYSTEM ===");
```

Exercise 4.3: Survival logic

The storm has passed, but you can feel its effects in every corner of the shelter. During the night, the strain of the storm put every system to the test: the generator keeps cutting out and the supplies are running out faster than expected.

This morning, while you check the damage with your field notebook in hand, the radio picks up a broken transmission. You don't know whether it comes from survivors, from a trap, or from plain interference, but the message suggests that someone might need help.

Your mission: experience has taught you that in situations like this you can't rely on a single signal, so you need to evaluate several conditions at once:

- Figure out whether you can go out: the perimeter has to be safe and, on top of that, either the weather has to be stable or the radio has to work.
- Figure out whether the shelter is in critical mode: the generator isn't working or you have fewer than 3 rations left.
- Figure out whether there's enough fuel to keep the generator running while you're away, based on how long the trip takes and how much fuel the generator uses per hour.
- Figure out whether the signal is worth investigating. All of these conditions have to be true:
 - You can go out.
 - The shelter isn't in critical mode.
 - There's enough fuel.
 - The zone of the signal is safe or, if it isn't, you're carrying at least 4 rations so you can hide for a few days.
- Print the four results to the console.

```
// Current conditions.
let generatorWorking: boolean = true;
let perimeterSafe: boolean = true;
```

```
let stableWeather: boolean = false;
let radioWorking: boolean = true;
let availableRations: number = 6;
let fuelLiters: number = 9;
let signalZoneSafe: boolean = false;

// Details of the trip to the source of the signal.
let hoursAway: number = 8;
let generatorConsumptionPerHour: number = 1; // Liters per hour.

console.log("=== DISTRESS SIGNAL ANALYSIS ===");
```

**Hint**

Remember the precedence of the different operators you use, and add parentheses if you need to make the order clear.

To be continued...

That's the end of the demo. You've packed your backpack, learned to label what's inside, told one kind of supply from another and done some arithmetic with them. That's enough to make a simple decision, but you still don't know what to do when the answer depends on several things at once, or how to repeat a routine without copying it twenty times. That comes right after this: conditionals and loops close the first block, and with them you step outside.

The full book goes much further. In the second block you form a group and take shelter in a power station, and along the way you learn functions, arrays, strings and dictionaries. In the third the station falls, there's an abandoned prison to take and defend, and object-oriented programming is what you need to organise that many people. The fourth block is the first winter in the prison: how to react when something fails, how to coordinate tasks that take time and run at the same time, and how to run drills before the horde arrives. Every block ends with a challenge, and at the back there are two reference appendices, one with the most common TypeScript errors and another on learning with an AI without letting it lead you astray.

If you've enjoyed what you've read so far, you can get the full book:

- [Paperback](#)
- [eBook](#)

Check the Amazon stores in other countries to find it there as well.